

סיכום הרצאות 2026 – מערכות הפעלה

מרצה: פרופסור דוד סרנה

© גיא יער-און

ייתכן שנפלו טעויות בעת כתיבת הסיכום, ולכן השימוש בסיכום על אחריותכם בלבד.

תוכן עניינים:

הרצאה 1: חזרה על מבנה מחשב, מבוא למערכות הפעלה **2**

הרצאה 1

מהי מטרת הקורס? עד היום התעסקנו ברמת תוכנית בודד ותהליך בודד: סינטקס של קוד, אלגוריתמים בהם משתמשים, כתיבת קוד נכון ויעיל וכו'. בקורס – אנחנו נתמקד בעולם בו פועל התהליך שלנו והאינטראקציה של אותו תהליך עם הסביבה, וכמובן נדון בכמה וכמה תהליכים. במהלך הקוד נדון בכמה וכמה שאלות:

- מה הן מערכות הפעלה?
- מה הן יודעות לעשות?
- כיצד מערכות הפעלה בנויות?
- מרכיבים עיקריים של מערכות הפעלה – תהליכים, Threads, CPU-Scheduling, וכו'.

נעיר מראש שבמהלך ההרצאות נדון בעקרונות כלליים של מערכות הפעלה – ולכן: בהרבה מהפעמים התשובה תהיה 'לא ניתן לדעת', שכן זה יהיה מאוד תלוי במערכת ההפעלה שתרוץ. **בתרגול:** נדון במערכת הפעלה ספציפית: Linux.

חזרה על מבנה מחשב:

- ארכיטקטורת Van-Neumann: משמשת כבסיס לרוב מערכות המחשבים המודרניות, היא מאוד מאוד פופולרית היום בעקבות יתרונותיה – אנחנו שומרים את הקוד ואת הdata שלנו באותו מקום בזיכרון, מה שהופך את התכנון והיישום של תוכנה וחומרה לפשוטים יותר. כמו כן, הארכיטקטורה נתמכת ע"י רוב שפות התכנות והכלים המודרניים. עם הזמן – התגלו בשנים האחרונות מגבלות בארכיטקטורה זו.
מגבלותיה במערכות מודרניות:
- 1. מחשוב מקבילי – מתאימה יותר למחשוב סדרתי, בעוד מערכות מודרניות דורשות יותר יכולת של תכנות מקבילי עם כמה וכמה מעבדים בו זמנית.
- 2. אבטחת מידע – העובדה שהקוד והנתונים נשמרים באותו מקום בזיכרון הוא פתח עבור "פולשים" שיוכלו לגנוב לי קוד.

במערכת ישנם 4 מודולים מרכזיים:

1. Memory – זיכרון: מאחסן את כל ההוראות ואת כל הdata של התוכנית שלנו. יש לנו בו את Control-Unit שאחראי על הרצת התוכנית שלנו ואת הAlu: אחראי על פעולות חשבון ולוגיקה לפי דרישות התוכנית וכן יש לנו בו את הIO לתקשורת עם "העולם החיצון".
- הזיכרון הוא אוסף של הרבה מאוד תאי זיכרון, כאשר גודל כל תא הוא קבוע. יחידת הזיכרון הבסיסית היא bit ותא זיכרון לרוב הוא byte בודד. הוא יקר ביחס לדיסקים וכדומה.
- הזיכרון עליו אנחנו מדברים נקרא RAM (Random Access Memory): מדובר בזמן גישה זהה לכל תא זיכרון. מדוע זה טוב לנו? כי זה גורר שלא אכפת לנו היכן התוכנית תשב בזיכרון – זמן הגישה זהה.
- WORD:** אוסף כל כמה ביטים (בהתאם לארכיטקטורה של המעבד), מדובר ביחידת המידע שמועבדת בפעולה בודדת. זהו גודל שלרוב זהה לגודל הרגיסטרים ולBUS. לרוב מדובר ב8 ביטים.
- לכל תא זיכרון ישנה כתובת:** את הכתובת אנחנו צריכים על מנת שנדע היכן הנתון נמצא בזיכרון. את הכתובות מייצגים באמצעות מחרוזת של ביטים. בהינתן n ביטים, נוכל לתמוך בזיכרון בגודל של $2^n - 1$ כתובות. מדובר בכמות המקסימלית של זיכרון שנוכל לשים במערכת שלי, כיוון שלא נוכל לייצג עוד זיכרון בשיטה זו.

סיכום מערכות הפעלה | © גיא יער-און

- את גודל הזיכרון מייצגים בבייטים – KB, MB, GB, TB וכו'.
- Hz הוא מס' הגישות לזיכרון שאנחנו יכולים לעשות בשנייה. כלומר 1 Hz הוא מחזור (Cycle) אחד בשנייה. מהו סייקל? יחידת העבודה הקטנה ביותר של זיכרון. הזיכרון יכול לבצע פעולה בכל רגע נתון. ככל שמס' הסייקלים גדול יותר פר שנייה הזיכרון עובד מהר יותר כי הוא מספיק לבצע יותר פעולות.
- מהירות הוא הנתון שאומר כמה סייקלים הזיכרון מבצע בשנייה אחת.

במקום לענות על השאלה: "כמה סייקלים יש בשנייה", נוכל לחשב כמה שניות לוקח סייקל אחד בודד:

$$\text{Time Per Cycle} = 1/\text{Hz}$$

לצורך ההמחשה: אם משהו קורה פעמיים בשנייה, כלומר 2 Hz, אז די ברור שכל פעם כזו (סייקל) לוקח חצי שנייה. כלומר, $\frac{1}{2}$.

פעולות על הזיכרון:

מערכת הזיכרון מופעלת באמצעות:

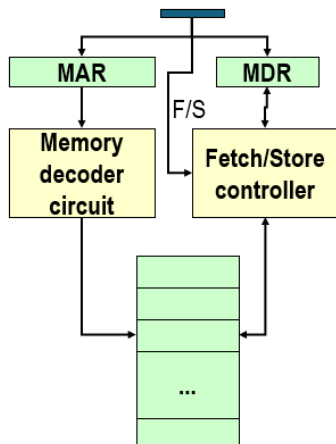
1. רגיסטר כתובות זיכרון – MAR – בעת שליפה ישמש לקרוא את הערך שאנחנו קוראים מהזיכרון.
2. רגיסטר נתוני זיכרון – MDR – ישמש לקרוא את הערך שנרצה לכתוב בזיכרון.
3. אות לשליפה/אחסון – אות חשמלי על מנת להבין האם הפעולה הבאה תהיה שליפה או אחסון.

- **שליפה – Fetch:** שליפת עותק של תוכן בתא הזיכרון עבור הכתובת שצוינה. מדובר בפעולה לא הרסנית כי היא משמרת את הערך בתא הזיכרון.

1. טוענת את הכתובת לתוך MAR

2. מבצע Decode לכתובת הזו: יחידת הזיכרון מפענחת את הכתובת שבMAR כדי לאתר פיזית את התא המתאים בזיכרון, היא תעתיק את התוכן של תא הזיכרון לתוך MDR
- אחסון – Store:** אחסון הערך שצוין לתוך תא הזיכרון עבור הכתובת שצוינה. פעולה הרסנית, דורסת את הערך הקודם בתא הזיכרון.

1. טוען את הכתובת לתוך MAR: המעבד מציב שם את כתובת התא אליו הוא רוצה לכתוב.
2. טוען את הערך לתוך MDR: המעבד מציב שם את הערך החדש שהוא רוצה לאחסן.
3. מבצע Decode של הכתובת לMAR: יחידת הזיכרון מפענחת את הכתובת בMAR על מנת לדעת להיכן לכתוב.
4. מעתיק את התוכן של MDR לתוך תא זיכרון עם הכתובת הספציפית שנבחר וכך מחליף את המידע הישן.



מודול הInput/Output (IO):

מטפל בהתקנים שמאפשרים למערכת המחשב לתקשר וליצור אינטראקציה עם העולם החיצון – לתקשר וליצור אינטראקציה עם העולם החיצון (מסך, מקלדת), לאחסן מידע.

לכל התקן של IO ישנו התקן בשם **IO CONTROLLER**: הבעיה היא שמהירות התקני IO נמוכה מאוד ביחס למהירות הRAM – ולכן בכל שימוש בIO נעכב את כל המערכת, לכן הפתרון הוא שימוש בהתקן זה. מדובר במעבד ייעודי שכולל **רכיב זיכרון קטן (IO Buffer)**: יש בו לוגיקת בקרה לניהול התקן הקלט/פלט: הוא ינהל עבורנו בצורה עצמאית את האינטראקציה עם המכשיר עצמו. למשל: הזזת זרוע הדיסק. היתרון: כיוון שהוא עצמאי, אפשר לבצע את הפעולות שלו במקביל למעבד שלי שמבצע פעולות של התוכנית. בסיום ניהול האינטראקציה

סיכום מערכות הפעלה | © גיא יער-און

עם device, ה-CONTROLLER יודיע לי על כך. ישנם כמה וכמה Controller's: אחד ייחודי לכל סוג של מכשיר. למשל – ישנו Device Controller עבור המסך שיכול לנהל כמה וכמה מסכים.

מודל ה-ALU:

מיועד לבצע פעולות מתמטיות, לוגיות. במחשבים של היום ה-ALU משולב כבר בתוך ה-CPU. הוא מורכב ממעגלים לביצוע הפעולות האריתמטיות/לוגיות, רגיסטרים לאחסון תוצאות חישוב הביניים, BUS שמחבר בין השניים.

מודל ה-Control Unit (יחידת הבקרה):

- התוכנית מאוחסנת בזיכרון כפקודות בשפת מכונה (לאחר שהקומפיילר הפך אותם לפקודות בשפת מכונה).
- תפקידה של יחידת הבקרה הוא לבצע תוכניות על ידי חזרה על השלבים הבאים:
 1. שליפה – Fetch: שליפת הפקודה הבאה לביצוע מהזיכרון
 2. פענוח – Decode: קביעת הפעולה שיש לבצע
 3. ביצוע – Execute: ביצוע הפעולה ע"י שליחת אותות מתאימים ל-ALU, לזיכרון ולתתי המערכות של הקלט/פלט. **תמיד שלב זה מתבצע מבחינת הקורס שלנו.**

הוא ממשיך לבצע את 3 הפעולות הללו, אחת אחרי השנייה: עד לפקודת עצירה.

פקודות בשפת מכונה:

פקודה בשפת מכונה מורכבת מ:

1. קוד פעולה: מציין איזו פקודה עלינו לבצע.
2. שדות כתובת – מציינים את כתובות הזיכרון של הערכים שעליהם הפקודה שלנו תפעל.

Opcode (8 bits)	Address 1 (16 bits)	Address 2 (16 bits)
00001001	0000000001100011	0000000001100100
קוד פעולה 9	כתובת זיכרון 99	כתובת זיכרון 100

לדוגמה: add x,y: אנחנו צריכים לקרוא את כתובת הזיכרון של x ואז את הכתובת של y וכן יש את ה-OPCODE של הפקודה. וזה יראה כך:

PC (Program counter) – רגיסטר ששומר את הכתובת של הפקודה הבאה בתור לביצוע. בעת שפקודה נשלפת מהזיכרון, הוא מתעדכן כדי להצביע על הפקודה הבאה בזיכרון.

IR (Instruction Register) – רגיסטר שמחזיק את הפקודה עצמה שנשלפה כרגע מהזיכרון, שומר את ה-OPCODE שעשינו לו Fetch מתוך הזיכרון.

Instruction Decoder: מפענח הפקודות. יחידה לוגית שמקבלת את הפקודה מה-IR ומפענחת אותה. הוא מתרגם את הקוד הבינארי של הפקודה לאותות חשמליים שמפעילים את הרכיבים הרלוונטיים. הוא הרכיב החומרתי שמופעל בעת ביצוע Decode לפקודה.

מבנה מערכת המחשב:

ניתן לחלק את מערכת המחשב לארבעה מרכיבים:

- אפליקציות (Applications): מגדירים כיצד יבוצע השימוש במשאבי המערכת על מנת לפתור בעיות חישוביות של משתמשים. היא יכולה להיות קומפיילר, משחק וידאו, אתר וכו'.
- משתמשים: לא בהכרח מדובר באנשים – גם מחשבים מרוחקים הם משתמשים. למשל עבור web server ישנם מחשבים שפונים אליו כבקשות ומוגדרים כמשתמשים.

סיכום מערכות הפעלה | © גיא יער-און

- מערכת ההפעלה: נמצאת מעל החומרה. היא שולטת על ומתאמת את השימוש במשאבי החומרה בין התהליכים והמשאבים השונים. מדובר על מערכות שיש בהן הרבה מאוד תהליכים שרצים – ואיננו יודעים מי מבין התהליכים יותר חשוב או יקר. עלינו רק לשלוט ולנהל אותם.
- חומרה (Hardware): ה"ברזלים". אילו משאבי מחשב בסיסיים יש לנו? CPU, Memory, IO Devices וכו'.

הגדרה: מערכת הפעלה היא תכנית אשר משמשת כחוצץ בין המשתמש של מערכת המחשב והחומרה של מערכת המחשב. מערכת ההפעלה נמצאת במכונות, במכשירים ביתיים, בטלפונים, מחשבים אישיים ותעשייתיים וכו'. מטרותיה: הרצת תוכניות המשתמש בצורה קלה, הפיכת השימוש במחשב לנוח יותר ושימוש יעיל יותר בחומרת המחשב.

נשים לב: מערכת ההפעלה מוגדרת כתוכנית גם כן – ולכן דורשת משאבים גם בעצמה. כלומר: גם כשלכאורה המחשב נראה שלא עושה כלום, המעבד מריץ קוד של מערכת ההפעלה.

שאלה. האם יכולה להיות מערכת מחשב ללא מערכת הפעלה? (כלומר, אם אפשר לקנות מחשב ולא להתקין עליו מערכת הפעלה).

תשובה. בתכלס, כן, אבל. זה ייצור עבורנו הרבה מאוד בעיות:

- נצטרך לכתוב תוכניות בקוד מכונה.
- לא נוכל להריץ יותר מתוכנית אחת בו זמנית.
- נצטרך לטפל בעצמנו בכל המקרים והתגובות – שגיאות, תזוזת עכבר, לחיצה על מקשים וכו'.
- נצטרך לנהל אינטראקציה עם החומרה בעצמנו.

מתי כן נראה את זה? במערכת סגורה – שבה אין אינטראקציה עם האדם, שהופך את התנהגות התוכנית ללא דטרמיניסטית. הרי, אם המערכת סגורה וידועה והכל בה דטרמיניסטי אז אין לנו צורך במערכת הפעלה (שדורשת הרבה משאבים).

סוגי קטגוריות מחשוב:

- **Supercomputer:** מחשב על. מחשב נמצא בחזית יכולת החישוב, במיוחד במהירות החישוב. מחשב מאוד יקר אותו מדינות/אוניברסיטאות קונות.
- **Mainframe:** מחשבים חזקים (מבחינת אמינות, זמינות וביצועים). משמשים בעיקר ארגונים גדולים ליישומים חשובים – כמו רישום אוכלוסייה, ניהול מלאי וכדומה. מהירות פחות גבוהה משל מחשב העל, אך עדיין גבוהה.
- **Personal Computer (PC):** כל מחשב רב תכליתי שגודלו, יכולותיו ומחירו המקורי הופכים אותו לשימושי עבור משתמשים פרטיים. אתה מוכר אותו עבור אדם ואינך יודע מה הוא יעשה בו – אם הוא ישתמש בו עבור מוזיקה בנטפליקס, או שיכתוב בו קוד.
- **Handheld:** דומה ל-PC, מכשיר מחשב אך עם סוללה שניתן לתפעול ביד אחת. הוא כולל בדרך כלל מסך תצוגה עם קלט של מגע. (כמו טלפון סלולרי).

לכל מערכת הפעלה יש מטרות ועיצוב שונים. למשל:

- **Mainframe:** הפוקוס של מערכת ההפעלה יהיה על ניצול מקסימלי של משאבי חומרה – פחות נוחות המשתמש, יותר ניצול מקסימלי של החומרה.
- **PC:** תמיכה מקסימלית בהרצת תוכניות של המשתמש – נוח עבור המשתמש.

UI-User Interface: המעטפת הוויזואלית והאינטראקטיבית שדרכה המשתמש מתקשר עם המכונה (כמו כפתורים, תפריטים ועיצוב המסך).

סיכום מערכות הפעלה | © גיא יער-און

- **Handheld**: שיהיה נוח להריץ אפליקציות, כי אין מקלדת/עכבר. נרצה גם ביצועים טובים פר יחידת ניצול של הסוללה.

בבירור – הנוחות מגיעה על חשבון היעילות. כמו תמיד, נאלץ לוותר על חלק מהיעילות עבור הנוחות ולוותר קצת על הנוחות עבור היעילות.

תפקידיה של מערכת ההפעלה:

- **Resource allocator**: בדומה לממשלה, שמקצה כמה מהתקציב ילך לכל תחום (חינוך/בריאות וכו'), מערכת ההפעלה מנהלת את המשאבים, היא מחליטה במצבים של קונפליקט על מנת שהשימוש במשאבים יהיה יעיל והוגן.
- **Control program**: שולטת על ההרצה של התוכניות השונות על מנת למנוע שגיאות, ולמנוע שימוש מוטעה במערכות המחשב.

תפקידיה חשובים מאוד בעיקר כאשר ישנם כמה משתמשים שמחברים לאותו mainframe.

שאלה. האם צייר, סוליטייר וכו' נחשבים כחלק ממערכת ההפעלה?

תשובה. אין תשובה אחת. יש כאלו שסוברים שכן – כל מה שהגיע בהתחלה כחלק מדיסק ההתקנה של מערכת ההפעלה הוא חלק ממערכת ההפעלה, ויש כאלו שסוברים שלא, והם לא חלק ממערכת ההפעלה.

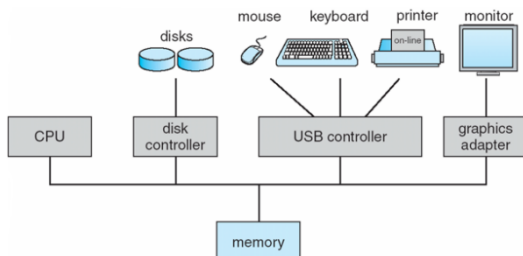
הגדרה: התוכנית/תהליך שרץ כל הזמן במערכת המחשב נקרא kernel. כל שאר התהליכים שרצים ע"פ דרישה מקוטלגים כ system programs (תוכניות שהגיעו יחד עם מערכת ההפעלה). למשל – הצייר מקוטלג כך. הוא הגיע עם מערכת ההפעלה, אך לא רץ כל הזמן. לעומת זאת application programs אלו דברים כמו chrome, הורדנו למחשב אך זה לא הגיע עם מערכת ההפעלה. כאשר אפליקציה רוצה לקרוא קובץ, להפעיל מצלמה, להקצות זיכרון – kernel מטפל בזה. הקרנל נותן יכולות בסיסיות, ותוכניות המערכת הופכות אותן למשהו נוח לשימוש.

Middleware frameworks: מדובר בשכבת ביניים שמטרתה היא לתת למפתחי אפליקציות שירותים מוכנים מבלי שיצטרכו להתעסק ישירות בפרטים הנמוכים של הקרנל. זה עדיין יעבור פונקציונאליות של הקרנל.

הפעלת המחשב: נניח וקנינו מחשב חדש, עליו כבר מותקנת מערכת ההפעלה. לחצנו על כפתור ההדלקה. מה קורה? ברגע שהדלקנו את המחשב ישנו תהליך של reboot. עולה **reboot program**: מדובר בתוכנית דטרמיניסטית שסגורה על עצמה ויודעת להתקדם קדימה ללא עזרה מהמשתמש – היא מאוחסנת בEPROM (רכיב חשמלי, לא נמחק בניגוד ל RAM, בעת שנסגור את המחשב ונדליק יום למחרת הוא יישאר שם). תוכנית זו מאתחלת את כל מרכיבי המערכת – תוכן הזיכרון, הרגיסטרים של המעבד וכו'. בנוסף: היא זו שמעלה את kernel של מערכת ההפעלה ומעבירה אליה את השליטה. בנקודה זו – מערכת ההפעלה הופכת לעצמאית, כל הקוד שלה טעון ומרכיביה מאותחלים. ואז – מערכת ההפעלה לא עושה כלום: היא מחכה, עד שיהיה אירוע כלשהו: שהמשתמש יפעיל את העכבר, יזיז משהו וכו'.

מעבדים:

מערכת ההפעלה מאופיינת ע"י מעבד (אחד או יותר, כיום יש כמה וכמה) Device 1 controllers: כמו Disk controllers controller וכו'. כל אחד מהם אחראי על סוג מסוים של devices. הם מחוברים באמצעות common bus שמאפשר להם גישה לזיכרון משותף, כך:



כל device controller אחראי על סוג מסוים של device. המחשב לא יודע כיצד לדבר ישירות עם המנוע של הארד דיסק, ולכן הוא מדבר עם הבקר והבקר יודע כיצד לנהל את המכשיר הספציפי שמוטאם לו.

סיכום מערכות הפעלה | © גיא יער-און

המידע מ ואל device מנוהל באמצעות Local Buffer: המעבד מעביר מידע מ/אל הזיכרון הראשי אל הלוקאל באפר. כיוון שהתקני IO איטיים מאוד בהשוואה למעבד, המידע לא יעבור ישירות מהדיסק למעבד. הלוקאל באפר הוא זיכרון מקומי קטן. המידע מהמכשיר נאסף קודם כל בבאפר של הבקר, כשהמידע מוכן הוא מועבר אל הזיכרון הראשי, ומשם למעבד (ולהפך). במקום שהמעבד ישב וישאל כל הזמן "סיימת? סיימת?" – הוא בונה על כך שבעת שהdevice controller יסיים הוא ישלח interrupt: לכן הוא יכול להמשיך לבצע פעולות אחרות תוך כדי.

ברוב המערכות נמצא לפחות מעבד אחד מסוג **general purpose**: מעבד שיועד לעשות הכל, הוא לא נבנה בצורה שנותנת יתרון ליישום מסוים – הוא נבנה בצורה כללית ויועד לבצע את הכל. לצד מעבד זה, מוסיפים מעבדים נוספים מסוג **special purpose**: למען מטרה מסוימת. למשל – GPU (מעבד עבור גרפיקה). לצד המעבדים הנוספים, תמיד נהיה חייבים את המעבד הכללי.

מערכות **Multiprocessors** הן מערכות בהן יש יותר ממעבד אחד – ישנה תקשורת בין המעבדים, שחולקים את אותו bus ולעיתים גם את הזיכרון. מה היתרונות של מערכת שכזו?

- מהירות, כמובן.
- כלכלי: אך למה שלא נקח מעבד פי 2 יותר מהיר, במקום לקחת 2 מעבדים? זה מגיע כמובן מבחינה כלכלית – לרוב ליצור מעבד מהיר פי 2, הרבה יותר יקר מאשר ליצור 2 מעבדים במהירות שקולה.
- בעת שיש תקלה, אם יש לנו רק מעבד אחד תקלה במעבד משביתה את המערכת כולה. בעת שיש כמה מעבדים, הם יכולים לכפות על המעבד שהתקלקל – המהירות תרד, אך המערכת לא תושבת.

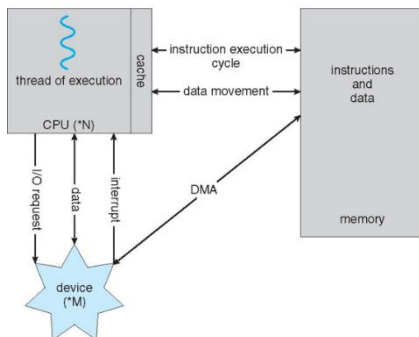
ישנן שתי ארכיטקטורות של ריבוי מעבדים:

- **Asymmetric Multiprocessing**: משימות מסוימות מוקצות למעבדים מסוימים בלבד. בפרט, יתכן כי מעבד יחיד יהיה אחראי על טיפול בכל interrupts במערכת ואולי אף יבצע פעולות קלט/פלט.
 - **Symmetric Multiprocessing**: מתייחס לכל רכיבי העיבוד במערכת באופן זהה – אפשר לפנות לכל המעבדים בעת דרישה.
- בכל פעם שנאמר בקורס core: הכוונה למעבד. (זה לא כך בפועל, אך זה הדיון במסגרת הקורס).

- **Clustered systems**: דומה לריבוי מעבדים, אך בריבוי מעבדים ישנם כמה מעבדים שחולקים אותו לוח אם ואותו זיכרון: כאן מדובר במערכות נפרדות (בזיכרון, לוח אם) שעובדות יחד. הן חולקות אחסון משותף דרך רשת שנקראת SAN.

היתרון הגדול הוא הזמינות – גם אם אחת המכונות קורסות, השירות לא יקרוס. בדומה, ישנן 2 שיטות לגישה זו:

1. **Asymmetric Clustering**: ישנה מכונה אחת שהיא במצב "Hot-StandBy": היא לא עושה כלום, והיא רק מחכה אם אחת השרתים יקרוס: היא תכנס במקומו.
2. **Symmetric Clustering**: כל המכונות עובדות ומריצות אפליקציות במקביל, אם אחת נופלת הן מתחלקות בעומס שלה. זה יעיל יותר כמובן כי כל החומרה מנוצלת.



איך מחשב מודרני פועל? יש לו מעבד – שמדבר גם עם הזיכרון, וגם עם device. בזיכרון – יש לנו גם פקודות, וגם דטה. המעבד יהיה זה שייקח דטה מהזיכרון, ויעביר אותו אל device: באמצעות device controller.

אם יש בקשה של device מהמעבד לבצע פקודות IO הוא ישלח בקשה: IO REQUEST, לשם כך לפעמים יהיה צריך להעביר data בין CPU לdevice. בסיום – device ישלח interrupt, עליה נרחיב כעת:

Interrupt: פסיקה – מדובר באות חשמלי שמתקבל במעבד מרכיב החומרה (דיסק, עכבר וכו'). נוצר ע"י device. הוא מעביר את השליטה ל **interrupt service routine (ISR)**. זו הדרך של device לומר "סיימתי": זו הדרך שלנו לעצור את מה שעושה המעבד. המעבר שעצר, יטפל כעת בinterrupt וכשהוא יסיים נוכל לחזור לעשות מה שעשינו קודם. הוא מחייב שמירה של ה כתובת של ה interrupted instruction ושל תוכן הרגיסטרים (על מנת שנוכל לחזור בהמשך למצב שבו היינו – עם ערכי הרגיסטרים שהיו לפני ביצוע ה interrupt). להכל יהיה interrupt: בכל פעם שה device controller יסיים משהו – הוא ישלח. נבחין כי בעת לחיצה של מקש במקלדת – ה device controller יטפל בכל התהליך – יזהה איזה מקש במקלדת הוקש וכו', ורק בסוף הוא יבצע interrupt.

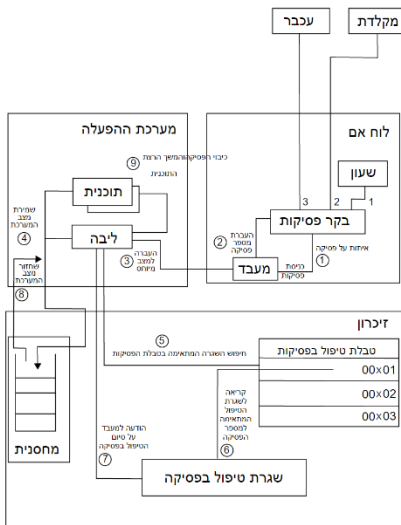
אם נסכם את התהליך, נעזר בתמונה משמאל להבנת התהליך:

שלב א': זיהוי – בלוח האם: התקן חיצוני (מקלדת/עכבר) שולח אות חשמלי לבקר הפסיקות. הבקר מעביר את האות אל המעבד. בקר הפסיקות שולח למעבד את ה"זהות" של הפסיקה (מספר הפסיקה. למשל: "פסיקה מספר 2 הגיעה דרך המקלדת")

שלב ב': החלפת הקשר – במערכת ההפעלה: המעבד מפסיק את ריצת התוכנית הרגילה ועובר ל kernel mode, על מנת שיוכל לגשת למשאבי מערכת מוגנים. כמו כן, המערכת שומרת את הנתונים הנוכחיים של התהליך (רגיסטרים, כתובות וכו') בתוך המחסנית על מנת שנוכל לחזור אליהם אחר כך ללא איבוד מידע.

שלב ג': איתור וביצוע – בזכרון: חיפוש בטבלת הפסיקות: המעבד נגש לטבלת וקטורי הפסיקות בזכרון ומחפש את הכתובת של הקוד שמתאים למס' הפסיקה שהתקבל בשלב 2. המעבד קופץ לכתובת שבטבלה ומצא, ומריץ את שגרת הטיפול בפסיקה (ISR), זהו הקוד שבאמת מטפל בלחיצת המקלדת או בתזוזת העכבר.

שלב ד': חזרה לשגרה – הודעה על סיום, המערכת מסיימת ומעדכנת את המעבד שהטיפול הושלם. שחזור מצב המערכת – שולפת מהמחסנית את הנתונים ששמרה ומחזירה אותם לרגיסטרים. המעבר חוזר למצב User Mode וממשיך להריץ את התוכנית בדיוק מהנקודה שבה נעצרה.



תרגול 1